Experiment 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A **Smart Contract** is a program stored on a blockchain that runs automatically when specific conditions are fulfilled. It is designed to enforce agreements without the need for intermediaries, making transactions more reliable and efficient. These contracts are typically written in Solidity and executed on the Ethereum Virtual Machine, which ensures that the logic is carried out exactly as programmed.

Smart contracts are a fundamental part of the Ethereum ecosystem, enabling secure and automated digital interactions.

Significance of Smart Contracts in Ethereum

- **Automation:** Executes predefined actions automatically without manual intervention
- **Transparency:** All contract logic and transactions are publicly visible on the blockchain
- **Security:** Once deployed, the code cannot be altered, reducing the risk of tampering
- **Cost Efficiency:** Eliminates the need for third-party intermediaries
- **Trustless Operation:** Parties can interact directly without relying on mutual trust

Smart contracts act as the backbone of decentralized applications (DApps) and decentralized finance (DeFi), supporting a wide range of blockchain-based services.

Crowdfunding Platform Overview

A blockchain-based crowdfunding platform leverages smart contracts to remove centralized control and improve trust among participants. Instead of relying on a third party, all operations are handled through automated contract logic.

In this system, two smart contracts are used:

- **CrowdfundingContract.sol:** Responsible for creating and managing fundraising campaigns, as well as tracking contributions
- **DonorContract.sol:** Handles donor-related actions and interacts with the main contract using a Solidity interface

This architecture ensures transparency, security, and a decentralized approach to fundraising, where all transactions are recorded on the blockchain and executed without intermediaries.

Tools Used:

- **Solidity** — High-level language for writing smart contracts on Ethereum.
- **Remix IDE** — Browser-based IDE for writing, compiling, deploying, and testing smart contracts using a built-in Remix VM.

Implementation:

CrowdfundingContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

contract CrowdfundingContract {

    struct Campaign {
        uint256 campaignId;
        string title;
        uint256 goalAmount;
        uint256 raisedAmount;
        uint256 deadline;
        address payable creator;
        bool isActive;
    }

    uint256 public campaignCount = 0;
    mapping(uint256 => Campaign) public campaigns;

        event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount,
address creator);
    event FundsReceived(uint256 campaignId, uint256 amount);

    function createCampaign(
        string memory _title,
        uint256 _goalAmount,
        uint256 _durationDays
    ) public {
        campaignCount++;
        uint256 deadline = block.timestamp + (_durationDays * 1 days);
        campaigns[campaignCount] = Campaign(
            campaignCount,
            _title,
            _goalAmount,
            0,
            deadline,
            payable(msg.sender),
            true
```

```
        );
        emit CampaignCreated(campaignCount, _title, _goalAmount, msg.sender);
    }

    function receiveDonation(uint256 _campaignId, uint256 _amount) external {
        Campaign storage c = campaigns[_campaignId];
        require(c.isActive, "Campaign is not active");
        c.raisedAmount += _amount;
        emit FundsReceived(_campaignId, _amount);
    }

    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    ) {
        Campaign storage c = campaigns[_campaignId];
        return (
            c.campaignId,
            c.title,
            c.goalAmount,
            c.raisedAmount,
            c.deadline,
            c.creator,
            c.isActive
        );
    }

    function closeCampaign(uint256 _campaignId) external {
        campaigns[_campaignId].isActive = false;
    }
}
```

DonorContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;
    function getCampaign(uint256 _campaignId) external view returns (
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}
contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
```

```
        address donor;
        uint256 amount;
        uint256 timestamp;
    }
    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256
amount);
    constructor(address _crowdfundingAddress) {
        crowdfundingContractAddress = _crowdfundingAddress;
    }
    function donate(uint256 _campaignId, uint256 _amount) public {
        donationCount++;
        donations[donationCount] = Donation(
            donationCount,
            _campaignId,
            msg.sender,
            _amount,
            block.timestamp
        );
        ICrowdfundingContract(crowdfundingContractAddress)
            .receiveDonation(_campaignId, _amount);

        emit DonationMade(donationCount, _campaignId, msg.sender, _amount);
    }
    function getDonation(uint256 _donationId) public view returns (
        uint256, uint256, address, uint256, uint256
    ) {
        Donation storage d = donations[_donationId];
        return (d.donationId, d.campaignId, d.donor, d.amount, d.timestamp);
    }
    function getCrowdfundingContract() public view returns (address) {
        return crowdfundingContractAddress;
    }
}
```

Compilation:

Compiled | Home | CrowdfundingContract.sol | DonorContract.sol

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
//MAHVISH SIDDIQUI

contract CrowdfundingContract {

    struct Campaign {
        uint256 campaignId;
        string title;
        uint256 goalAmount;
        uint256 raisedAmount;
        uint256 deadline;
        address payable creator;
        bool isActive;
    }

    uint256 public campaignCount = 0;
    mapping(uint256 => Campaign) public campaigns;

    event CampaignCreated(uint256 campaignId, string title, uint256 goalAmount, address creator);
    event FundsReceived(uint256 campaignId, uint256 amount);

    function createCampaign(    infinite gas
        string memory _title,
        uint256 _goalAmount,
        uint256 _durationDays
    ) public {
        campaignCount++;
        uint256 deadline = block.timestamp + (_durationDays * 1 days);
        campaigns[campaignCount] = Campaign(
            campaignCount,
```

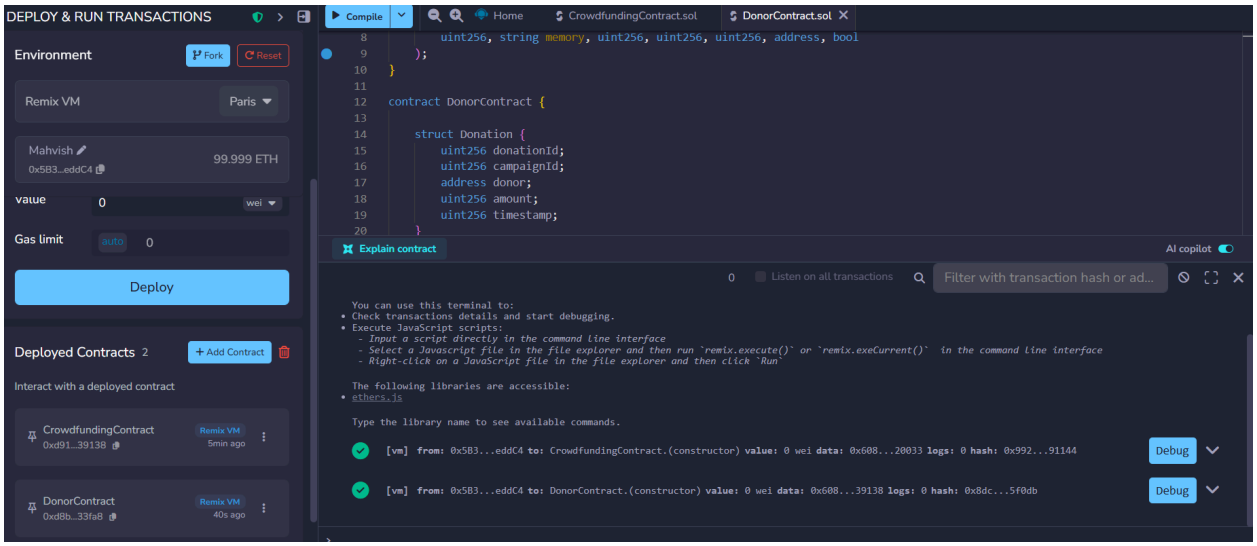
Explain contract | AI copilot

Compiled | Home | CrowdfundingContract.sol | DonorContract.sol

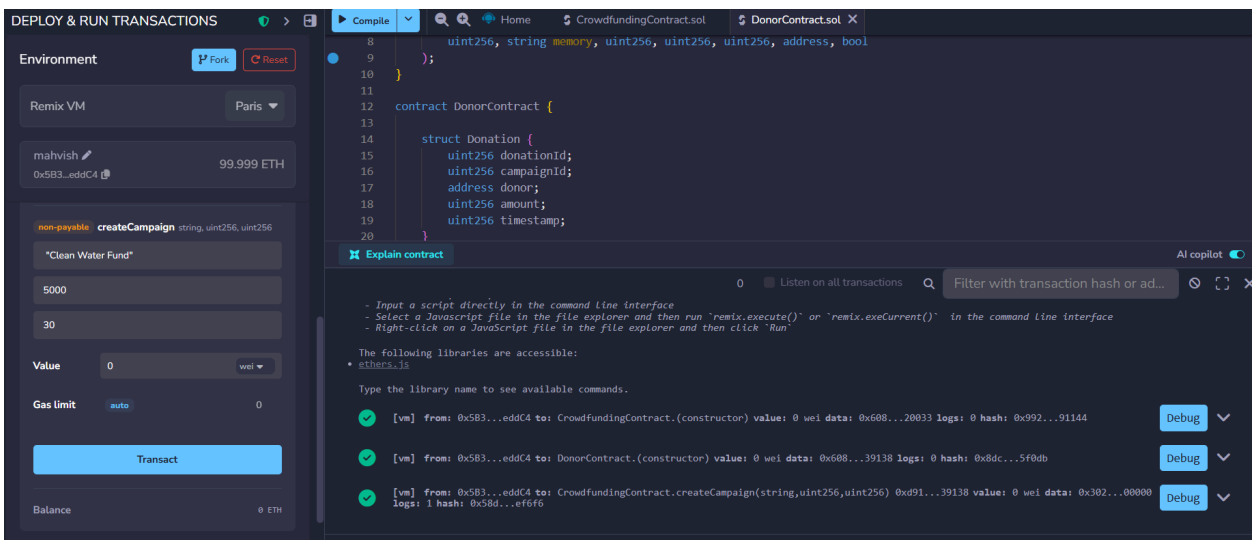
```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
//MAHVISH SIDDIQUI

interface ICrowdfundingContract {
    function receiveDonation(uint256 _campaignId, uint256 _amount) external;    - gas
    function getCampaign(uint256 _campaignId) external view returns (    - gas
        uint256, string memory, uint256, uint256, uint256, address, bool
    );
}

contract DonorContract {

    struct Donation {
        uint256 donationId;
        uint256 campaignId;
        address donor;
        uint256 amount;
        uint256 timestamp;
    }

    uint256 public donationCount = 0;
    mapping(uint256 => Donation) public donations;
    address public crowdfundingContractAddress;

    event DonationMade(uint256 donationId, uint256 campaignId, address donor, uint256 amount);

    constructor(address _crowdfundingAddress) {    infinite gas 340400 gas
        crowdfundingContractAddress = _crowdfundingAddress;
    }
```

Explain contract | AI copilot

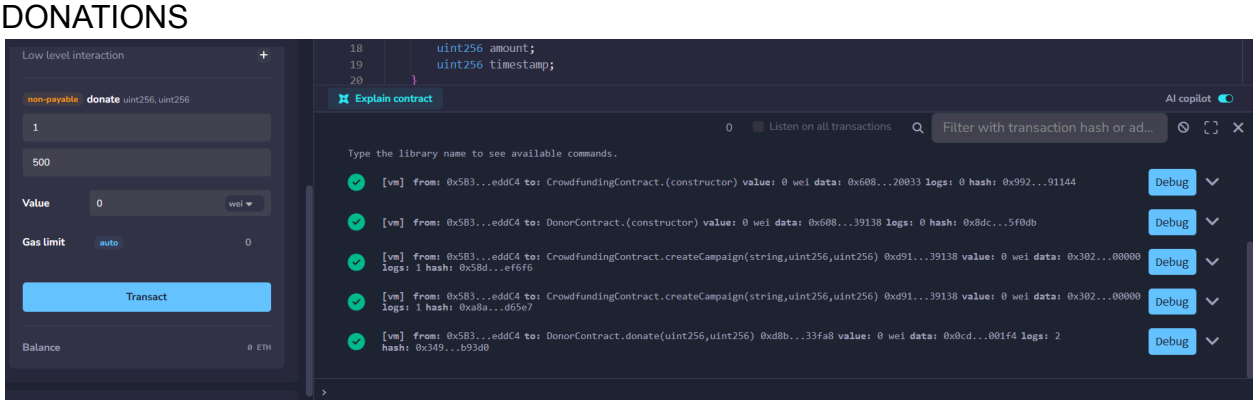
DEPLOYMENT:

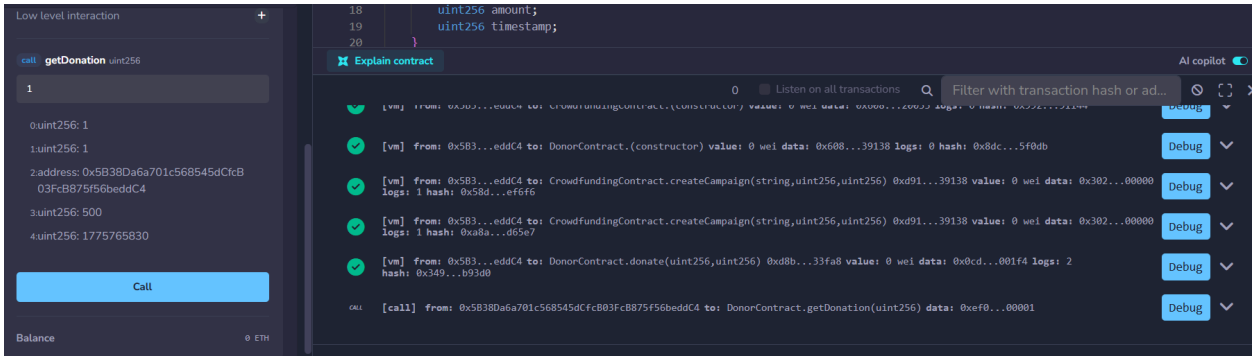


TRANSACTIONS:



DONATIONS





Conclusion:

This experiment helped in understanding how smart contracts can automate and secure a crowdfunding platform on the Ethereum blockchain. Using Solidity and Remix IDE, two contracts — CrowdfundingContract and DonorContract — were designed, deployed, and tested. The CrowdfundingContract manages campaign creation and tracks donations, while the DonorContract handles donor interactions. Together they demonstrate a trustless, transparent, and intermediary-free fundraising system.